

```
import pandas as pd
```

```
accident_data=pd.read_csv('accident_prediction_india.csv')
display(accident_data.head())
print(accident_data.columns)
```

	State Name	City Name	Year	Month	Day of Week	Time of Day	Accident Severity	Number of Vehicles Involved	Vehicle Type Involved	Number of Casualties	...	Road Type	Road Condition	C
0	Jammu and Kashmir	Unknown	2021	May	Monday	1:46	Serious	5	Cycle	0	...	National Highway	Wet	
1	Uttar Pradesh	Lucknow	2018	January	Wednesday	21:30	Minor	5	Truck	5	...	Urban Road	Dry	
2	Chhattisgarh	Unknown	2023	May	Wednesday	5:37	Minor	5	Pedestrian	6	...	National Highway	Under Construction	
3	Uttar Pradesh	Lucknow	2020	June	Saturday	0:31	Minor	3	Bus	10	...	State Highway	Dry	
4	Sikkim	Unknown	2021	August	Thursday	11:21	Minor	5	Cycle	7	...	Urban Road	Wet	

5 rows × 22 columns

```
Index(['State Name', 'City Name', 'Year', 'Month', 'Day of Week',
      'Time of Day', 'Accident Severity', 'Number of Vehicles Involved',
      'Vehicle Type Involved', 'Number of Casualties', 'Number of Fatalities',
      'Weather Conditions', 'Road Type', 'Road Condition',
      'Lighting Conditions', 'Traffic Control Presence', 'Speed Limit (km/h)',
      'Driver Age', 'Driver Gender', 'Driver License Status',
      'Alcohol Involvement', 'Accident Location Details'],
      dtype='object')
```

```
accident_data.info()
accident_data['Speed Limit (km/h)'].describe()
accident_data['Driver Age'].describe()
```



```
<class 'pandas.core.frame.DataFrame'>
```

Data Cleaning, Removing Non-Important Columns & Handling Null Values

Data columns (total 22 columns):

```
#Time to Category
def get_time_category(time_str):
    hour = int(str(time_str).split(':')[0])

    if 6 <= hour < 18:
        return 'Day'
    else :
        return 'Night'

accident_data['Time Category'] = accident_data['Time of Day'].apply(get_time_category)
accident_data=accident_data.drop('Time of Day',axis=1)
print(accident_data['Time Category'].value_counts())

#Speed
accident_data['Speed Category']=pd.cut(accident_data['Speed Limit (km/h)'],bins=[0,30,75,350],labels=['Low','Moderate','High'])
accident_data=accident_data.drop('Speed Limit (km/h)',axis=1)
display(accident_data['Speed Category'].value_counts())

#Age Mapping
accident_data['Age Group'] = pd.cut(
    accident_data['Driver Age'],
    bins=[12, 30, 60, 100],
    labels=['Young', 'Adult', 'Senior'],
    include_lowest=True
)
accident_data=accident_data.drop('Driver Age',axis=1)
display(accident_data['Age Group'].value_counts())

#columns selection
selected_col=[
    'State Name',
    'Time Category',
    'Weather Conditions',
    'Road Type',
    'Road Condition',
    'Lighting Conditions',
    'Traffic Control Presence',
    'Speed Category',
    'Vehicle Type Involved',
    'Age Group'.
```

```
selected_col = ['Driver Gender',
                'Driver License Status',
                'Alcohol Involvement',
                'Accident Severity'
                ]

from sklearn.preprocessing import LabelEncoder
label_encoders = {} #In this we store mapping of label encoders
for col in selected_col:
    if accident_data[col].dtype=='object' or accident_data[col].dtype=='category':
        le=LabelEncoder()
        accident_data[col]=le.fit_transform(accident_data[col])
        label_encoders[col]=le

label_encoders
```

```
Time Category
Night      1511
Day        1489
Name: count, dtype: int64
```

```
count

Speed Category
-----
High          1484
Moderate      1478
Low           38
```

```
dtype: int64
count
```

```
Age Group
-----
Adult    1683
Young     748
Senior    569
```

```
dtype: int64
{'State Name': LabelEncoder(),
 'Time Category': LabelEncoder(),
 'Weather Conditions': LabelEncoder(),
 'Road Type': LabelEncoder(),
 'Road Condition': LabelEncoder(),
 'Lighting Conditions': LabelEncoder(),
 'Traffic Control Presence': LabelEncoder(),
 'Speed Category': LabelEncoder(),
 'Vehicle Type Involved': LabelEncoder(),
 'Age Group': LabelEncoder(),
 'Driver Gender': LabelEncoder(),
 'Driver License Status': LabelEncoder(),
 'Alcohol Involvement': LabelEncoder(),
 'Accident Severity': LabelEncoder()}
```

```
accident_data['Traffic Control Presence']=accident_data['Traffic Control Presence'].fillna('Unknown')
accident_data['Driver License Status']=accident_data['Driver License Status'].fillna('Unknown')
filtered_accident_data = accident_data[selected_col]
```

```
display(filtered_accident_data.info())
filtered_accident_data.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   State Name                            3000 non-null   int64
1   Time Category                         3000 non-null   int64
2   Weather Conditions                    3000 non-null   int64
3   Road Type                             3000 non-null   int64
4   Road Condition                        3000 non-null   int64
5   Lighting Conditions                   3000 non-null   int64
6   Traffic Control Presence              3000 non-null   int64
7   Speed Category                        3000 non-null   int64
8   Vehicle Type Involved                 3000 non-null   int64
9   Age Group                             3000 non-null   int64
10  Driver Gender                         3000 non-null   int64
11  Driver License Status                 3000 non-null   int64
12  Alcohol Involvement                   3000 non-null   int64
13  Accident Severity                     3000 non-null   int64
dtypes: int64(14)
memory usage: 328.3 KB
None
```

	State Name	Time Category	Weather Conditions	Road Type	Road Condition	Lighting Conditions	Traffic Control Presence	Speed Category	Vehicle Type Involved	Age Group	Driver Gender	Driver License Status	Alcohol Involvement
0	11	1	2	0	3	0	2	2	3	1	1	2	1
1	29	1	2	2	1	3	2	0	5	0	1	2	1
2	5	1	1	0	2	1	2	0	4	2	0	2	0
3	29	1	3	1	1	0	1	0	1	0	0	1	1
4	25	0	1	2	3	3	2	0	3	2	1	2	0

Next steps: [Generate code with filtered_accident_data](#) [New interactive sheet](#)

Separating Features And Target

```
target=filtered_accident_data['Accident Severity']
features=filtered_accident_data.drop('Accident Severity',axis=1)
print(features.shape)
print(target.shape)
```

```
(3000, 13)
(3000,)
```

Model Training Using Logistic Regression our baseline Model

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(
    features,target,
    test_size=0.2,
    random_state=42
)

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,classification_report,confusion_matrix

lr_model=LogisticRegression()
lr_model.fit(X_train,y_train)
y_pred=lr_model.predict(X_test)

print(accuracy_score(y_test,y_pred))
print(classification_report(y_test,y_pred))
print(confusion_matrix(y_test, y_pred))
```

```
0.30833333333333335
```

	precision	recall	f1-score	support
0	0.27	0.14	0.18	203
1	0.32	0.49	0.39	202
2	0.31	0.30	0.30	195
accuracy			0.31	600
macro avg	0.30	0.31	0.29	600
weighted avg	0.30	0.31	0.29	600

```
[[ 28 108 67]
 [ 38 99 65]
 [ 39 98 58]]
```

Model Training Using RandomForest Classifier

```
from sklearn.ensemble import RandomForestClassifier
rf_model=RandomForestClassifier(n_estimators=200, random_state=42)
rf_model.fit(X_train,y_train)
rf_predictions= rf_model.predict(X_test)

print(accuracy_score(y_test,rf_predictions))

print(classification_report(y_test,rf_predictions))
importance = pd.Series(rf_model.feature_importances_, index=features.columns)
importance.sort_values(ascending=False)
```

0.365

	precision	recall	f1-score	support
0	0.35	0.33	0.34	203
1	0.39	0.46	0.42	202

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.